腾讯云（CSIG）后台开发面经整理

- 腾讯云（CSIG）后台开发面经整理 —— 八股问答
  - 0. 面试风格总结
  - 1. 操作系统
  - 2. 计算机网络
  - 3. MySQL
  - 4. Redis
  - 5. 分布式 / 云原生（腾讯云重点）
  - 6. Go（腾讯云很多组用 Go）
  - 7. C++（简历写 C++ 时）
  - 8. Java（腾讯云智常考，简历写 Java 时）
  - 9. AI 相关（2026 新增高频）
  - 10. 智力 / 场景题
  - 来源
- 腾讯云（CSIG）后台开发面经整理 —— 手撕题
  - 一、题目清单（按出现频率排序）
  - 二、ACM 模式模板
  - 三、重点题题解
  - 四、复习建议

腾讯云（CSIG）后台开发面经整理 —— 八股问答

> 整理时间：2026-10。素材来自牛客网、面灵、卡码笔记、小林coding等 2025-2026 年的腾讯云 / CSIG / 腾讯云智面经（来源见文末）。 题目后的【频】表示在多篇面经里反复出现。

0. 面试风格总结

- 一面：通常先**手撕 1~2 题**（ACM 模式或限时 20~30 分钟），然后是计算机基础（OS、网络）、语言（Go/C++/Java 看简历）、MySQL/Redis，最后拷打项目。
- 腾讯云方向偏**分布式、多机架构、容器/K8s**：etcd、Raft、iptables、容器网络、“单机改多机怎么做”。
- 2026 年新增高频：**AI Coding**（Cursor/Codex 怎么用、怎么保证 AI 代码正确）、**MCP 与 Skills 区别**、LangChain/LangGraph、Agent 状态流转。
- 二面偏**场景题和追问**：“TCP 一定比 UDP 慢吗”、“大表查询要考虑什么”、“设计 0~10 万整数集合”。
- 面试官会追问到底层，答“是什么”之后要准备“为什么”和“怎么实现”。

---

1. 操作系统

1.1 进程、线程、协程区别【频】

- **进程**：资源分配的基本单位，独立地址空间；切换要换页表、刷 TLB，开销大；隔离性好。

- **线程**：CPU 调度的基本单位，共享进程地址空间、fd 等；独有栈、寄存器、PC、TLS；切换只换寄存器和栈。
- **协程**：用户态调度的轻量线程，由运行时/程序切换，不陷入内核，栈小（Go 初始 2KB 可增长），可创建百万级。
- Linux 中线程也是 `task_struct`，**调度器按线程（task）调度**。
- Python 有协程：`async/await` + `asyncio`（以及早期 generator `yield`）。

1.2 为什么进程切换比线程切换开销大？上下文包含什么？【频】

- 上下文：通用寄存器、PC、栈指针、程序状态字、内核栈；进程还包括页表基址（CR3）、内存映射。
- 进程切换要**切换页表 → TLB 失效**，后续访存大量 TLB miss / cache miss，这是主要间接开销。
- 切换流程：中断/系统调用进入内核 → 保存当前任务寄存器到内核栈/task_struct → 调度器选下一个任务 → 切换 mm（页表）→ 恢复寄存器 → 返回用户态。

1.3 进程间通信方式【频】

管道（匿名/命名 FIFO）、消息队列、**共享内存（最快，无需内核拷贝，但要自己同步）**、信号量、信号、Socket（含 Unix Domain Socket）、mmap 文件映射。 - 共享内存为什么能互相访问：不同进程的虚拟页通过页表映射到**同一块物理页**。 - 管道实现：内核中一块环形缓冲区（pipe_inode_info），两个 fd 分别指向读端/写端。

1.4 用户态与内核态

- CPU 特权级区分（x86 Ring3 / Ring0），用户态不能执行特权指令、访问内核内存。
- 切换途径：系统调用（`syscall` 指令）、异常（缺页）、外部中断。
- 系统调用流程：用户库函数把调用号放寄存器 → `syscall` 陷入 → 保存用户上下文 → 查系统调用表执行 → 恢复上下文 `sysret` 返回。

1.5 死锁【频】

- 四条件：互斥、占有并等待、不可剥夺、循环等待。
- 解决：固定加锁顺序（破坏循环等待）、一次性申请所有资源、`try_lock` + 超时回退、银行家算法（避免）、检测后杀进程/回滚（解除）。
- 排查：gdb attach 看 `thread apply all bt`、`pstack`；Java 用 `jstack`；Go 用 `pprof goroutine`。

1.6 OS 底层如何实现锁？

- 硬件原子指令：CAS（x86 `lock cmpxchg`）、`xchg`、`test-and-set`、LL/SC。
- 自旋锁：CAS 循环；互斥锁（futex）：用户态先 CAS 抢，失败才 `futex_wait` 进内核睡眠，释放时 `futex_wake`。
- **A 加锁后 B 不加锁直接改变量能否成功？能。** 锁是“君子协定”（advisory），只约束同样去拿锁的线程，不保护内存本身。

1.7 进程虚拟地址空间（从低到高）

代码段 .text → 数据段 .data → BSS → 堆（向上增长，brk）→ 内存映射区（mmap、动态库）→ 栈（向下增长）→ 内核空间。 - 堆 vs 栈：栈编译器自动管理、快、大小有限（默认 8MB）；堆手动/GC 管理、大、分配慢、有碎片。

1.8 虚拟内存 / overcommit

- 512M 机器申请 1G：`malloc` 只分配虚拟地址，是否成功看 `vm.overcommit_memory`（0 启发式、1 总允许、2 严格）。真正写入时缺页才分配物理页，不够则 swap 或触发 **OOM Killer**。

1.9 孤儿进程 / 僵尸进程 / 守护进程

- 孤儿：父进程先退出，子进程被 init/systemd（PID 1）收养。
- 僵尸：子进程退出但父进程未 `wait()`，PCB 残留占 PID。解决：父进程 `wait/waitpid`、处理 `SIGCHLD`、或杀掉父进程让 init 回收。
- 守护进程：脱离终端在后台运行（`fork` → `setsid` → 再 `fork` → `chdir /` → 关闭 fd）。

1.10 文件系统

- inode：存文件元数据（权限、大小、时间、数据块指针），不含文件名；目录项存“文件名 → inode 号”。
- 硬链接：多个目录项指向同一 inode，删一个不影响；不能跨文件系统/不能链目录。软链接：独立 inode，内容是目标路径，目标删了就失效。
- ext4 寻址：extent 树（起始块 + 长度），替代 ext2/3 的直接/间接块指针。

1.11 IO 多路复用 select / poll / epoll【频】

	select	poll	epoll
fd 上限	1024	无	无
每次调用	拷贝全部 fd 集合	同左	只 `epoll_ctl` 注册一次
就绪检测	线性遍历 O(n)	O(n)	回调放入就绪链表 O(1)

- epoll 内部：**红黑树**存监听 fd，**双向链表**存就绪 fd。
- LT（水平触发）：只要可读就一直通知；ET（边缘触发）：状态变化才通知一次，必须**非阻塞 fd + 循环读到 EAGAIN**。

1.12 Linux 常用命令【频】

- 端口占用：`ss -tlnp | grep 8080`、`netstat -anp | grep 8080`、`lsof -i:8080`
- 查进程：`ps -ef | grep xxx`、`pgrep xxx`、`top/htop`
- 文件内找字符串：`grep -rn "str" dir/`；找文件：`find / -name "a.txt"`
- 进程所在目录：`ls -l /proc/<pid>/cwd`、`/proc/<pid>/exe`
- 内存/磁盘：`free -h`、`df -h`、`du -sh`；负载：`uptime`、`vmstat`、`iostat`
- 发信号：`kill -9 pid`、`kill -15 pid`
- 抓包与调试：`tcpdump`、`strace`、`gdb`、`perf`

---

2. 计算机网络

2.1 三次握手，为什么不能两次【频】

- SYN(seq=x) → SYN+ACK(seq=y, ack=x+1) → ACK(ack=y+1)。

- 不能两次：①防止**历史重复的 SYN** 让服务端建立无效连接，浪费资源；②双方都要确认对方的初始序列号（ISN）。
- 不需要四次：服务端的 ACK 和 SYN 可以合并在一个包里发。
- 半连接队列（SYN 队列）与全连接队列（accept 队列）；SYN Flood 防御：`tcp_syncookies`、增大 backlog、减少 SYN+ACK 重试次数。

2.2 四次挥手，ACK 和 FIN 能否合并【频】

- 被动方收到 FIN 时可能还有数据没发完，所以先回 ACK，数据发完再发 FIN → 一般四次。
- **能合并**：被动方没有待发数据且开启延迟确认时，ACK 和 FIN 合成一个包，抓包会看到“三次挥手”。
- TIME_WAIT 持续 2MSL：①保证最后的 ACK 能到达（丢了对端会重发 FIN）；②让旧连接残留的报文在网络中消失，不干扰新连接。
- TIME_WAIT 过多：`tcp_tw_reuse`、改用长连接、让客户端主动关闭；CLOSE_WAIT 过多 = **应用没调用 close**，是代码 bug。

2.3 TCP 是全双工还是半双工?

全双工，两端可同时收发，各自维护发送/接收窗口和序列号。

2.4 TCP vs UDP，使用场景【频】

- TCP：面向连接、可靠、有序、字节流、流量控制和拥塞控制、头部 20 字节起。
- UDP：无连接、不可靠、面向报文、无拥塞控制、头部 8 字节、支持广播/多播。
- TCP 用于 HTTP/1.1、HTTP/2、SSH、FTP、SMTP、MySQL；UDP 用于 DNS、DHCP、QUIC/HTTP3、音视频、游戏同步。
- **UDP 监听 443，TCP 能否再监听 443？能**，端口按协议区分，五元组里包含协议号（HTTP/3 就是 UDP 443 + TCP 443）。

2.5 TCP 一定比 UDP 慢吗？（CSIG 场景追问）

不一定。在低丢包、长连接、大流量传输时，TCP 窗口打满后吞吐接近带宽，内核和网卡对 TCP 有 TSO/GRO 等硬件卸载，效率可能比用户态可靠 UDP 更高。TCP 慢主要在：建连握手、慢启动、丢包后拥塞窗口减小、队头阻塞。 - 下载有时快有时慢：拥塞控制根据网络状况动态调整 cwnd（慢启动 → 拥塞避免 → 丢包降窗），同时受对端接收窗口、链路竞争影响。下载开始慢、后来快 = 慢启动。 - 滑动窗口不是按“硬带宽”设置：发送窗口 = min(接收窗口 rwnd, 拥塞窗口 cwnd)。

2.6 TCP 可靠传输如何实现【频】

序列号/确认号、超时重传（RTO）与快速重传（3 个重复 ACK）、SACK、校验和、流量控制（接收窗口）、拥塞控制（慢启动、拥塞避免、快速恢复；BBR）。

2.7 粘包

TCP 是字节流，没有消息边界。解决：定长消息、分隔符、**长度字段 + 消息体（最常用）**。UDP 面向报文，不会粘包。 - 场景题：服务端循环每次发 1 字节，客户端循环每次用新的 4 字节 buffer read → 一次 read 可能读到 1~4 字节，不保证读满 4 字节，必须根据返回值处理；如果按 4 字节解析会读到错乱的数据。

2.8 如何用 UDP 实现可靠传输

序列号 + ACK + 超时重传 + 滑动窗口 + 拥塞控制，做在应用层。例子：QUIC、KCP（用带宽换延迟，更激进的重传）。

2.9 HTTP 相关

- HTTP/1.1：长连接 keep-alive、管道化（存在队头阻塞）；HTTP/2：二进制分帧、多路复用、HPACK 头部压缩、服务端推送；HTTP/3：基于 QUIC/UDP，解决 TCP 层队头阻塞，0-RTT/1-RTT 建连，支持连接迁移。
- 报文长度表示：`Content-Length`，或 `Transfer-Encoding: chunked`（以长度为 0 的块结尾）。
- HTTPS：TLS 握手 → 证书校验（CA 签名链）→ 用非对称加密/ECDHE 协商对称密钥 → 之后用对称加密传输。
- GET vs POST、401（未认证）vs 403（无权限）、GET vs HEAD（HEAD 只返回响应头）。
- 应用层：用 TCP 的有 HTTP(1/2)、FTP、SMTP、SSH；用 UDP 的有 DNS（大响应或区域传输时用 TCP）、DHCP、SNMP、QUIC。

2.10 输入 URL 到页面展示

DNS 解析（浏览器缓存 → hosts → 本地 DNS → 根 → 顶级域 → 权威）→ TCP 握手 → TLS 握手 → 发 HTTP 请求 → 服务端处理（负载均衡/网关/业务）→ 返回响应 → 浏览器解析渲染。

2.11 NAT / iptables / VLAN（腾讯云偏好）

- NAPT：把内网 (IP, port) 映射成公网 (IP, 新port)，记在映射表里，响应根据目的端口查表转回内网主机。
- iptables：基于内核 **netfilter** 钩子（PREROUTING/INPUT/FORWARD/OUTPUT/POSTROUTING），按表（filter/nat/mangle/raw）和链匹配规则；Docker 端口映射、K8s Service（kube-proxy iptables 模式）都靠 nat 表的 DNAT/SNAT。
- VLAN：802.1Q 在以太帧里加 12bit VLAN ID，把一个物理二层网络划分为多个广播域，实现隔离。
- 容器网络：veth pair 一端在容器 netns、一端接宿主机网桥 docker0，出网走 iptables MASQUERADE（SNAT）。

---

3. MySQL

3.1 InnoDB 索引结构，为什么用 B+ 树【频】

- B+ 树：非叶子节点只存 key 和指针，扇出大（一页 16KB 能放上千个指针），3~4 层就能存千万级数据，即 3~4 次磁盘 IO；叶子节点通过双向链表连接，适合范围查询。
- vs B 树：B 树非叶子节点也存数据，扇出小、树更高；做范围查询要中序遍历。
- vs 哈希：不支持范围查询和排序；vs 红黑树/二叉树：树太高。
- 索引为什么能加速：把 O(n) 的全表扫描变为 O(log n) 的有序查找，同时减少了随机 IO。

3.2 索引类型 / 聚簇与非聚簇 / 回表【频】

- 按结构：B+Tree、Hash、全文索引；按逻辑：主键、唯一、普通、联合、前缀索引。
- 聚簇索引：叶子节点存整行数据（InnoDB 主键）；二级索引叶子节点存主键值，需要**回表**。MyISAM 都是非聚簇（叶子存数据行的地址）。
- 覆盖索引：要查的列都在索引中，不用回表。
- 最左前缀原则；索引下推 ICP。

- 索引失效：违反最左前缀、对索引列使用函数或做运算、隐式类型转换、% 开头的 like、or 中有非索引列、!=/not in（视优化器而定）、区分度低时优化器选择全表扫描。
- **商品类型字段适合建索引吗？** 区分度低（类型就几种），单独建索引效果差，可作为联合索引的一部分（比如 (type, create_time)）。

3.3 事务 ACID 与实现

- 原子性 → undo log；持久性 → redo log（WAL + 两阶段提交配合 binlog）；隔离性 → 锁 + MVCC；一致性是目标，靠前三者保证。

3.4 隔离级别【频】

读未提交（脏读）→ 读已提交（不可重复读）→ **可重复读（MySQL 默认）**（幻读，InnoDB 用 MVCC + Next-Key Lock 基本解决）→ 串行化。

3.5 MVCC【频】

- 隐藏列 trx_id、roll_pointer，通过 undo log 串成版本链。
- Read View：m_ids（活跃事务列表）、min_trx_id、max_trx_id、creator_trx_id。可见性规则：trx_id < min → 可见；trx_id ⩾ max → 不可见；在 m_ids 中 → 不可见，否则可见。
- RC 每次 select 都生成 Read View；RR 在事务第一次 select 时生成并复用。
- 快照读走 MVCC；当前读（select ... for update、update、delete）加锁。

3.6 锁：行锁、间隙锁、Next-Key Lock【频】

- Record Lock 锁记录；Gap Lock 锁区间（开区间），防止插入；**Next-Key Lock = 记录锁 + 前面的间隙，左开右闭区间**，是 RR 下加锁的基本单位。
- 唯一索引等值查询命中时退化为记录锁；未命中时退化为间隙锁。
- 没走索引的 update 会锁全表（每条记录都加 next-key lock）。

3.7 大表 / 深分页 / 慢查询

- 慢查询：开慢查询日志 → EXPLAIN 看 type（ALL/index/range/ref/eq_ref/const）、key、rows、Extra（Using filesort / Using temporary / Using index）。
- 深分页 limit 1000000, 10：改用游标/记录上次的 id where id > last_id limit 10；或子查询先在覆盖索引上查出 id 再 join 回表。
- 大表：建合适的索引、只查需要的列、冷热数据分离、读写分离、分库分表（按用户 id 哈希/时间范围）。
- 主从延迟导致读不到最新数据：关键读走主库、写后短时间内读主库、半同步复制、根据 GTID/位点判断从库是否已同步。

---

4. Redis

4.1 为什么用 Redis 而不是 MySQL / 为什么快【频】

基于内存、单线程执行命令避免锁和上下文切换、IO 多路复用（epoll）、高效数据结构；6.0 起网络 IO 可以多线程，命令执行仍是单线程。

4.2 数据结构与底层实现

- String（SDS）、List（quicklist/listpack）、Hash（listpack/哈希表）、Set（intset/哈希表）、ZSet（listpack / **跳表 + 哈希表**）。
- 其他：Bitmap、HyperLogLog、GEO、Stream。
- 渐进式 rehash：两张哈希表，每次增删改查时迁移一个桶，避免一次性阻塞。

4.3 持久化

- RDB：bgsave fork 子进程，利用写时复制（COW）做快照；文件小、恢复快，但会丢失最后一次快照之后的数据。
- AOF：记录写命令，`appendfsync always/everysec/no`；AOF 重写压缩文件。
- 4.0 以后支持混合持久化（AOF 重写时把 RDB 写在文件头部）。

4.4 过期删除与内存淘汰

- 过期删除：惰性删除 + 定期删除。
- 淘汰策略：noeviction、allkeys-lru/lfu/random、volatile-lru/lfu/random/ttl。Redis 的 LRU 是**近似 LRU（随机采样）**。

4.5 高可用：主从、哨兵、Cluster【频】

- 主从同步：第一次全量同步（PSYNC → 主节点 bgsave 生成 RDB 发给从节点 + 发送期间缓冲的写命令）；之后增量同步（命令传播，断线重连时用复制积压缓冲区 `repl_backlog` + offset 续传）。
- 哨兵：监控、主观下线 / 客观下线（多数哨兵认可）、通过 Raft 方式选出领头哨兵来执行故障转移。
- Cluster：16384 个哈希槽，`CRC16(key) % 16384`，去中心化 Gossip 通信，每个分片主从；客户端会收到 MOVED/ASK 重定向。

4.6 缓存穿透 / 击穿 / 雪崩【频】

- 穿透（查询不存在的数据）：布隆过滤器、缓存空值。
- 击穿（热点 key 过期）：互斥锁重建、逻辑过期、热点 key 不设过期时间。
- 雪崩（大量 key 同时过期或 Redis 宕机）：过期时间加随机值、多级缓存、限流降级、Redis 高可用。

4.7 缓存与数据库一致性【频】

- 推荐 **Cache Aside：先更新 DB，再删缓存**（不是更新缓存）。
- 为什么不先删缓存再更新 DB：并发读会把旧值写回缓存。
- 删缓存失败：重试 + MQ，或者订阅 **binlog（Canal）** 异步删除；延迟双删。
- 想要强一致只能加分布式锁或读写串行化，代价很高。

4.8 分布式锁

- `SET key uuid NX PX 30000` 加锁；解锁用 Lua 脚本判断 value 是自己的再 DEL（保证原子性，防止误删别人的锁）。
- 任务执行时间超过锁的超时时间：看门狗（Redisson watchdog）定时续期。
- 主从切换会导致锁丢失：Redlock（向多数独立节点加锁），或改用 etcd/ZooKeeper 等 CP 系统。

4.9 布隆过滤器

k 个哈希函数映射到位数组；判断“不存在”一定准确，判断“存在”可能误判；不支持删除（Counting Bloom Filter 可以）。误判率由位数组大小 m、元素个数 n、哈希函数个数 k 决定。

---

5. 分布式 / 云原生（腾讯云重点）

5.1 CAP / BASE

- C 一致性、A 可用性、P 分区容错。出现网络分区时只能在 C 和 A 之间选。etcd/ZooKeeper 选 CP；Eureka、Redis Cluster 偏 AP。
- BASE：基本可用、软状态、最终一致性。

5.2 Raft【频】

- 角色：Leader / Follower / Candidate；任期 term。
- 选举：Follower 选举超时（随机 150~300ms）后变为 Candidate，term+1，发 RequestVote；获得多数票成为 Leader；投票限制是只投给日志至少和自己一样新的候选人。
- 日志复制：Leader 发 AppendEntries，日志被多数节点持久化后才提交（commit），然后应用到状态机。
- 脑裂：少数派那边的旧 Leader 无法获得多数确认，写不成功；网络恢复后看到更高的 term 自动退为 Follower。
- 加入新节点：先作为 learner 同步日志，追上后再通过 joint consensus 或单节点成员变更加入集群。

5.3 etcd

- **强一致（线性一致）**：基于 Raft；默认读是 linearizable read（通过 ReadIndex 实现），也可以配置 serializable read（读本地，可能读到旧数据）。
- MVCC：每次修改 revision 全局递增，boltdb（B+ 树）存储 `revision → kv`，内存 btree 索引存 `key → revisions`。
- **Watch 机制**：客户端通过 gRPC stream 注册 watch（可以指定起始 revision），服务端用 synced/unsynced watcherGroup 管理；有写入时匹配 watcher 并推送事件；客户端断线后用最后的 revision 续 watch，不丢事件。
- “为什么用 offset / revision”：断点续传、按版本增量同步，避免重复或丢失事件（和 Kafka 的 offset 道理相同）。
- 用途：服务注册发现（lease + keepalive）、配置中心、分布式锁、选主。

5.4 Kafka

- 高可用：分区多副本、ISR、`acks=all`、`min.insync.replicas>=2`。
- 不丢消息：生产者 acks=all + 重试 + 幂等；Broker 端多副本；消费者端处理完再手动提交 offset。
- 高吞吐：顺序写磁盘、page cache、零拷贝 `sendfile`、批量发送与压缩、分区并行。
- 顺序消费：同一个业务 key 发到同一分区，每个分区只由一个消费者消费。

5.5 Docker / K8s

- Docker 隔离：**Namespace**（PID、NET、MNT、UTS、IPC、USER）负责隔离，**cgroups** 负责资源限制，联合文件系统（overlayfs）负责镜像分层。
- 和虚拟机比：容器共享宿主机内核，启动是秒级、开销小，但隔离性弱；虚拟机有独立内核。
- K8s 核心：Pod、Deployment、Service、Ingress、ConfigMap；控制面包括 apiserver、etcd、scheduler、controller-manager；节点上有 kubelet、kube-proxy。
- 项目怎么部署：Dockerfile 构建镜像 → 推到镜像仓库 → K8s Deployment 滚动更新，或用 CI/CD 流水线。

5.6 单机改多机 / 分布式缓存设计（常见追问）

- 数据分片（一致性哈希 + 虚拟节点）、副本复制（主从或 Raft）、服务发现（etcd）、客户端路由或代理层、故障转移、热点 key 处理。
- 一致性哈希：节点和 key 都映射到哈希环上，key 顺时针找到第一个节点；增删节点只影响相邻的一段数据；用虚拟节点解决数据倾斜。

5.7 大文件中找访问量 Top 10 的词条（外部排序 / 分治）

哈希取模把大文件拆成 N 个小文件（同一个词一定落到同一个文件）→ 每个小文件用 HashMap 计数，再用大小为 10 的小顶堆取 Top10 → 合并 N 个结果再取 Top10。 - 同类题：1 亿个数取前 100（小顶堆）；海量 int 去重（bitmap，2^32 位 = 512MB）；1000 万个范围在 0~100 万的整数排序（计数排序）。

5.8 文件快速分发到 100 万台服务器

P2P（参考 BitTorrent / 迅雷 / 腾讯的 P2P 分发）：把文件切块，每台机器既下载又上传；或者树形分层分发。

---

6. Go（腾讯云很多组用 Go）

6.1 slice【频】

- 结构：`{ptr, len, cap}`；数组是值类型且长度属于类型的一部分，slice 是对底层数组的引用视图。
- 扩容（1.18+）：cap < 256 时翻倍；之后按 `newcap += (newcap + 3*256)/4` 平滑增长；最后按内存规格（size class）向上取整。
- 坑：多个 slice 共享底层数组时，append 可能覆盖别人的数据；作为函数参数传入后在函数里 append，如果发生扩容外部看不到。

6.2 interface 底层【频】

- `eface{_type, data}`（空接口）；`iface{tab *itab, data}`，itab 包含接口类型、动态类型和方法表。
- **经典坑：`var p *T = nil; var i interface{} = p; i != nil`**，因为 type 字段不为空。
- interface 可以用 == 比较：类型相同且值相同才相等；如果动态类型不可比较（比如 slice、map），会 panic。
- 1.18 以后 interface 可以作为类型约束（类型集合），例如 `~int | ~string`。

6.3 泛型

1.18 引入，支持类型参数 `func Map[T any, R any](s []T, f func(T) R) []R`，约束可以用 `comparable`、`constraints.Ordered`；实现方式是 GC Shape Stenciling + 字典。

6.4 map

- 底层是 hmap + buckets（每个 bucket 存 8 个 kv，有 tophash 和溢出桶），负载因子 6.5 时翻倍扩容，溢出桶太多时等量扩容，渐进式迁移。（Go 1.24 改为 Swiss Table 实现。）
- 并发读写会直接 fatal error（**无法 recover**）。解决：`sync.RWMutex`、`sync.Map`（read + dirty 双 map，适合读多写少）、分片加锁。
- 遍历顺序是随机的；key 必须是可比较类型。

6.5 channel

- hchan：环形缓冲区 buf、sendx/recvx、recvq/sendq 等待队列、一把 mutex（**互斥锁，不是读写锁**）。
- 无缓冲：发送和接收同步交接；有缓冲：缓冲区满时发送阻塞，空时接收阻塞。
- 已关闭的 channel：读会先读完剩余数据，再返回零值和 ok=false；写会 panic；重复关闭也会 panic。

6.6 GMP【频】

- G 是 goroutine，M 是系统线程，P 是逻辑处理器（数量 = GOMAXPROCS，持有本地运行队列，最多 256 个 G）。
- 新建的 G 优先放到当前 P 的 runnext/本地队列，本地队列满了就把一半转移到全局队列。
- G 进行阻塞系统调用时，M 和 P 解绑，P 去找别的 M 继续执行；网络 IO 由 netpoller 处理，不阻塞 M。
- 工作窃取（work stealing）；1.14 起基于信号的异步抢占。

6.7 GC

三色标记 + 混合写屏障（1.8），并发标记，STW 时间很短；根对象包括全局变量、各 G 的栈、寄存器。

6.8 其他

- defer 是 LIFO 执行；defer 在 return 赋值之后、函数真正返回之前执行；panic 时依次执行 defer，可以在 defer 中 recover。
- **子协程的 panic 不能被父协程 recover**，必须在子协程内部自己 recover。
- 逃逸分析：返回局部变量的指针、闭包引用、interface 装箱、对象太大、发送到 channel 等情况会逃逸到堆上，用 `go build -gcflags=-m` 查看。
- context：在调用链上传递取消信号、超时和请求范围的值。
- 等待多个协程完成：`sync.WaitGroup`、`errgroup`。

---

7. C++（简历写 C++ 时）

- **vector**：三个指针 begin/end/end_of_storage；扩容时 1.5 倍（MSVC）或 2 倍（GCC）重新分配内存并移动元素，原有迭代器全部失效；默认构造时容量为 0。
- **map / set** 是红黑树，**unordered_map** 是哈希表（链地址法），负载因子超过 1 时 rehash。
- list 为什么是双向链表：支持 O(1) 删除任意节点、双向迭代器（`--it`）、O(1) 的 `push_back`。
- 红黑树：节点非红即黑、根节点和叶子（NIL）都是黑色、红节点的子节点都是黑色、任意路径上的黑节点数量相同；最长路径 ≤ 最短路径的 2 倍，插入/删除最多旋转 2~3 次。
- **虚函数**：每个含虚函数的类有一张虚表（vtable），每个对象有一个虚表指针（vptr，在构造函数中设置）；通过 vptr 查表实现运行时多态。构造函数不能是虚函数（此时 vptr 还没初始化）；基类析构函数应该声明为虚函

数，否则通过基类指针 delete 派生类对象时派生类的析构不会执行。

- **new vs malloc**：new 是运算符，调用 operator new 分配内存并执行构造函数，返回类型化指针，失败抛 `bad_alloc`；malloc 是库函数，只分配内存，返回 `void*`，失败返回 NULL。
- **智能指针**：`unique_ptr` 独占所有权，只能移动；`shared_ptr` 使用引用计数（控制块，计数操作是原子的）；循环引用会导致内存泄漏，用 `weak_ptr` 打破。
- static：静态局部变量只初始化一次（C++11 起线程安全）、静态全局变量/函数限定在本文件可见、静态成员属于类本身。
- 内存对齐：CPU 按字长访问内存，对齐后访问效率高，有的平台不对齐会出错。
- 编译流程：预处理 → 编译 → 汇编 → 链接（产出 ELF 文件）。
- 空类默认生成：默认构造、析构、拷贝构造、拷贝赋值，C++11 还有移动构造和移动赋值。
- map 在循环中删除元素：`it = m.erase(it)`，避免迭代器失效。

---

8. Java（腾讯云智常考，简历写 Java 时）

- HashMap：数组 + 链表 + 红黑树（链表长度 ⩾ 8 且数组长度 ⩾ 64 时树化）；put 流程：计算 hash → 定位桶 → 桶为空直接插入 / 否则遍历链表或树 → 超过阈值 resize。
- ConcurrentHashMap：1.7 用 Segment 分段锁；1.8 用 CAS（桶为空时）+ synchronized 锁桶头节点。
- CAS 与 ABA：ABA 问题用版本号解决（`AtomicStampedReference`）。AQS：volatile state + CLH 双向队列。
- ThreadLocal：key 是弱引用，防止 ThreadLocal 对象无法被回收；value 是强引用，在线程池中必须调用 `remove()` 否则会内存泄漏；异步任务传递值用 `InheritableThreadLocal` / TTL。
- JVM 内存结构、OOM 排查：`jmap -dump` 导出堆 + MAT 分析；`jstack` 看线程；`top -Hp` 找到 CPU 高的线程。
- 线程池 7 个参数、4 种拒绝策略；Spring IOC/AOP、SpringBoot 自动配置原理（`@EnableAutoConfiguration` + `spring.factories` / `AutoConfiguration.imports`）。

---

9. AI 相关（2026 新增高频）

- **MCP（Model Context Protocol）**：Anthropic 提出的开放协议，让模型以统一方式调用外部工具和数据源（Client-Server 架构，提供 tools / resources / prompts 三类能力）。
- **Skills**：把某类任务的说明文档、脚本、模板打包，Agent 按需加载（渐进式披露：先只读描述，用到时再读全文），本质是“给模型的操作手册”。
- **怎么选**：需要访问外部系统、实时数据、有副作用的操作 → MCP；需要固定流程、领域知识、最佳实践 → Skills。两者可以组合（Skill 中指导模型调用 MCP 工具）。
- **LangChain vs LangGraph**：LangChain 偏线性的链式编排和组件库；LangGraph 用有向图（可以有环）和显式 State 描述多步骤/多 Agent 流程，支持循环、分支、持久化 checkpoint、人工介入。
- 多智能体之间传递消息：共享 State（黑板模式）、消息队列、函数调用 / handoff；状态流转用状态机或图的边条件来表达。
- **怎么保证 AI 写的代码是正确的**：需求拆小、逐步生成；人工 review；单元测试和集成测试（可以让 AI 先写测试）；静态检查和 lint；CI 门禁；灰度发布。业务问题排查依靠日志、链路追踪、监控指标和复现。

---

10. 智力 / 场景题

- AB 两地相距 10km，去程 30km/h，返程速度多少才能让往返平均速度达到 60km/h？**不可能**：平均 60 意味着总时间 20/60 = 1/3 h，而去程已经用了 10/30 = 1/3 h，返程时间必须为 0。
- rand5 实现 rand7（拒绝采样）：`x = 5*rand5() + rand5()` 得到 [0,24] 上的均匀分布，x < 21 时返回 x % 7，否则重来。rand5 实现 rand3：rand5 结果 < 3 时直接返回，否则重来。
- 64 匹马，8 条跑道，求最快的 4 匹：11 场（8 场分组赛 + 1 场冠军赛 + 筛选后再赛 2 场）。25 匹马 5 条跑道求前 3：7 场。
- 30 瓶水中有 1 瓶毒药、只有一天时间：5 只小白鼠（二进制编码，2^5 = 32 ⩾ 30）。
- 判断平面上的点能否构成凸多边形：先求凸包（Graham / Andrew 单调链），看凸包上的点数是否等于点的总数（并且没有点在凸包内部）。
- 设计一个集合存 0~10 万的整数，CRUD 和 foreach 尽量快：**bitmap 判断存在 + 稠密数组存元素 + 下标映射（sparse set）**，删除时把要删的元素和末尾元素交换再删末尾，所有操作 O(1)，foreach 只遍历实际存在的元素。（参考 LC 380 O(1) 插入删除随机获取。）

来源

- 牛客：腾讯后台开发一面-腾讯云-技术与产品服务（2026.4）https://www.nowcoder.com/feed/main/detail/c4aee8bae13d46db90b8183bd76bc6a6
- 牛客：腾讯云后台面经（2025.9）https://www.nowcoder.com/discuss/799948416796065792
- 牛客：C++ 后端暑期实习记录（含腾讯云一面）https://www.nowcoder.com/discuss/875014834968879104
- 牛客：腾讯 CSIG 一二面 https://www.nowcoder.com/discuss/872562616462278656
- 面灵：腾讯云 CSIG 后台开发一面 https://mj.mianlingai.com/interview/tencent-csig-backend-1/
- 面灵：腾讯云智后端一面 https://mj.mianlingai.com/interview/tencent-backend-first-round-2842627/
- 面灵：腾讯面经合集 https://mj.mianlingai.com/company/%E8%85%BE%E8%AE%AF/
- 卡码笔记：腾讯云面经 https://notes.kamacoder.com/interview/java/tencent-cloud-interview.html
- 小林coding：腾讯云智面经 https://www.xiaolincoding.com/backend_interview/internet_medium/tencent_cloud_intelligence.html
- 腾讯云开发者社区：往期腾讯面经汇总（Go）https://developer.cloud.tencent.com/article/2640241
- 技术栈：腾讯后端校招面经 https://jishuzhan.net/article/2047476014725529601
- 牛客：CSIG 腾讯云存储一/二面 https://www.nowcoder.com/feed/main/detail/5b657639cf014472a5aa3dca24dc72e5

腾讯云（CSIG）后台开发面经整理 —— 手撕题

> 来源：2025-2026 年牛客、面灵等平台的腾讯云 / CSIG / 腾讯云智面经（链接见《腾讯云面经-八股问答.md》文末）。 腾讯云手撕的特点：**ACM 模式（自己处理输入输出）**、限时 20~30 分钟、面试官会自己造测试用例跑；**链表题出现次数最多**，LRU 几乎每轮都可能出现。 代码用 C++（Go 版本的思路相同）。

一、题目清单（按出现频率排序）

#	题目	LeetCode	出处	核心思路
1	LRU 缓存（含并发、分布式版本追问）	146	腾讯云日常实习、PCG、CSIG 二面	哈希表 + 双向链表
2	K 个一组翻转链表 / **从尾部开始** K 个一组	25	CSIG 一面、秋招一面	分段反转；从尾部开始 = 先跳过 len%k 个节点
3	两两交换链表中的节点（2 个一组翻转）	24	腾讯云一面（限时 20 分钟）	同 25，k=2
4	合并 K 个升序链表	23	CSIG 一面	小顶堆 / 分治两两合并
5	合并两个有序链表（ACM 模式）	21	腾讯云一面	哑节点 + 双指针
6	最接近的三数之和	16	腾讯云一面	排序 + 双指针
7	在排序数组中查找元素的第一个和最后一个位置	34	腾讯云日常实习	手写 lower_bound / upper_bound
8	可被三整除的最大和	1262	腾讯云一面（2026.4）	状态 DP（余数 0/1/2）
9	消消乐（相邻重复字符消除）	1047 / 1209	腾讯云一面（2026.4）	栈
10	只出现一次的数字 II（其余数出现三次）	137	腾讯云一面	按位统计次数再模 3
11	编辑距离	72	CSIG 二面	二维 DP
12	用 Rand5 实现 Rand3 / Rand7	470	秋招一面	拒绝采样
13	两数之和	1	腾讯云智一面	哈希表
14	非递归快排	912	CSIG 实习一面	用栈模拟递归
15	阻塞队列，生产者-消费者模型	—	CSIG 云存储一面	mutex + 两个条件变量
16	大数乘法	43	腾讯二面	竖式乘法
17	判断点集能否构成凸多边形	—	CSIG 一面（只说思路）	凸包 / 叉积同号
18	O(1) 时间增删查 + 遍历的集合	380	CSIG 二面设计题	数组 + 哈希（或下标数组）
19	合并区间 / 对称二叉树 / 二叉树最大路径和 / 快速幂	56 / 101 / 124 / 50	腾讯其他 BG，顺手复习	—
20	交替打印奇偶数（多协程/多线程）	1116	腾讯后端日常实习	channel / 条件变量

建议额外刷（腾讯面试高频）：206 反转链表、92 反转链表 II、141/142 环形链表、160 相交链表、19 删除倒数第 N 个节点、143 重排链表、148 排序链表、215 第 K 大元素、3 无重复字符的最长子串、200 岛屿数量、300 最长递增子序列、102 层序遍历、236 最近公共祖先、20 有效括号、239 滑动窗口最大值。

二、ACM 模式模板

```
#include <bits/stdc++.h>
using namespace std;
struct ListNode { int val; ListNode* next; ListNode(int v): val(v), next(nullptr) {} };

// 输入一行数字，构造链表
ListNode* readList() {
    string line; getline(cin, line);
    stringstream ss(line); int x;
    ListNode dummy(0), *p = &dummy;
    while (ss >> x) p = p->next = new ListNode(x);
    return dummy.next;
}
void printList(ListNode* h) {
    for (; h; h = h->next) cout << h->val << (h->next ? " " : "\n");
}
```

三、重点题题解

1. LRU 缓存（LC 146）—— 必须能默写

```
class LRUCache {
    int cap;
    list<pair<int,int>> lst;                                    // 链表头部是最近使用的
    unordered_map<int, list<pair<int,int>>::iterator> mp;
public:
    LRUCache(int c): cap(c) {}
    int get(int k) {
        auto it = mp.find(k);
        if (it == mp.end()) return -1;
        lst.splice(lst.begin(), lst, it->second);           // O(1) 移到链表头部
        return it->second->second;
    }
    void put(int k, int v) {
        if (auto it = mp.find(k); it != mp.end()) {
            it->second->second = v;
            lst.splice(lst.begin(), lst, it->second);
            return;
        }
        if ((int)lst.size() == cap) { mp.erase(lst.back().first); lst.pop_back(); }
        lst.emplace_front(k, v);
        mp[k] = lst.begin();
    }
};
```

面试官常常要求手写双向链表（不能用 `std::list`），结构是 `prev/next` 指针 + 头尾哨兵节点，操作就是 `remove(node)` 和 `addToHead(node)`。 **追问：** - 并发安全：最简单是一把全局 mutex（注意 get 也会修改链表，所以不能用读写锁）；进一步做**分片**，按 `hash(key) % N` 分成 N 个 LRU，每个分片一把锁。 - 分布式：用一致性哈希把 key 路由到不同节点，每个节点维护本地 LRU；节点用主从或 Raft 复制，节点列表放在 etcd 里做服务发现。 - LRU-K / LFU（LC 460）：LFU 用 `key → (val, freq)` + `freq → 链表` + 维护 minFreq。

2. K 个一组翻转链表（LC 25）+ 从尾部开始的变种

```
ListNode* reverseKGroup(ListNode* head, int k) {
    ListNode dummy(0); dummy.next = head;
    ListNode* pre = &dummy;
    while (true) {
        ListNode* tail = pre;
        for (int i = 0; i < k; i++) { tail = tail->next; if (!tail) return dummy.next; }
        ListNode *cur = pre->next, *nxt = tail->next;
        // 头插法把 [cur, tail] 这一段反转
        ListNode *prev = nxt, *p = cur;
        while (p != nxt) { ListNode* t = p->next; p->next = prev; prev = p; p = t; }
        pre->next = tail;   // tail 变成这一段的头
        pre = cur;          // cur 变成这一段的尾
    }
}
// 从尾部开始 k 个一组：先求链表长度 n，跳过前 n%k 个节点，对剩下的部分调用 reverseKGroup
ListNode* reverseKGroupFromTail(ListNode* head, int k) {
    int n = 0; for (auto p = head; p; p = p->next) n++;
    int skip = n % k;
    if (skip == 0) return reverseKGroup(head, k);
    ListNode* p = head;
    for (int i = 1; i < skip; i++) p = p->next;
    p->next = reverseKGroup(p->next, k);
    return head;
}
```

LC 24 两两交换就是 `reverseKGroup(head, 2)`。

3. 合并 K 个升序链表（LC 23）

暴力解法是逐个合并，O(kN)；面试官会引导你写**小顶堆 O(N log k)** 或**分治 O(N log k)**。

```
ListNode* mergeKLists(vector<ListNode*>& lists) {
    auto cmp = [](ListNode* a, ListNode* b){ return a->val > b->val; };
    priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);
    for (auto l : lists) if (l) pq.push(l);
    ListNode dummy(0), *p = &dummy;
    while (!pq.empty()) {
        auto n = pq.top(); pq.pop();
        p = p->next = n;
        if (n->next) pq.push(n->next);
    }
    return dummy.next;
}
```

4. 合并两个有序链表（LC 21）

```
ListNode* merge(ListNode* a, ListNode* b) {
    ListNode dummy(0), *p = &dummy;
    while (a && b) {
        if (a->val <= b->val) { p->next = a; a = a->next; }
        else                  { p->next = b; b = b->next; }
        p = p->next;
    }
    p->next = a ? a : b;
    return dummy.next;
}
int main() { printList(merge(readList(), readList())); }
```

5. 最接近的三数之和（LC 16）

```
int threeSumClosest(vector<int>& a, int target) {
    sort(a.begin(), a.end());
    int best = a[0] + a[1] + a[2];
    for (int i = 0; i + 2 < (int)a.size(); i++) {
        int l = i + 1, r = a.size() - 1;
        while (l < r) {
            int s = a[i] + a[l] + a[r];
            if (abs(s - target) < abs(best - target)) best = s;
            if (s == target) return s;
            s < target ? l++ : r--;
        }
    }
    return best;
}
```

6. 查找第一个和最后一个位置（LC 34）—— 手写 lower_bound

```
// 返回第一个 >= x 的下标
int lowerBound(vector<int>& a, int x) {
    int l = 0, r = a.size();          // 左闭右开区间 [l, r)
    while (l < r) {
        int m = l + (r - l) / 2;
        if (a[m] < x) l = m + 1; else r = m;
    }
    return l;
}
vector<int> searchRange(vector<int>& a, int t) {
    int L = lowerBound(a, t);
    if (L == (int)a.size() || a[L] != t) return {-1, -1};
    return {L, lowerBound(a, t + 1) - 1};   // upper_bound(t) = lowerBound(t+1)
}
```

7. 可被三整除的最大和（LC 1262）

dp[r] 表示当前所有选择中，和模 3 余 r 的最大和。

```cpp
int maxSumDivThree(vector<int>& nums) {
    int dp[3] = {0, INT_MIN, INT_MIN};
    for (int x : nums) {
        int nd[3]; copy(dp, dp + 3, nd);
        for (int r = 0; r < 3; r++)
            if (dp[r] != INT_MIN) nd[(r + x) % 3] = max(nd[(r + x) % 3], dp[r] + x);
        copy(nd, nd + 3, dp);
    }
    return dp[0];
}
```

另一种解法是贪心：先求总和，如果余 1，就减去最小的一个余 1 的数，或者最小的两个余 2 的数（取两者中较小的）；余 2 的情况对称处理。

8. 消消乐（LC 1047 / 1209）

```cpp
// 1047：两个相邻的相同字符消除
string removeDuplicates(string s) {
    string st;
    for (char c : s) { if (!st.empty() && st.back() == c) st.pop_back(); else st.push_back(c); }
    return st;
}
// 1209：k 个相邻的相同字符消除，栈中存（字符，连续个数）
string removeDuplicatesK(string s, int k) {
    vector<pair<char,int>> st;
    for (char c : s) {
        if (!st.empty() && st.back().first == c) { if (++st.back().second == k) st.pop_back(); }
        else st.push_back({c, 1});
    }
    string r; for (auto& [c, n] : st) r.append(n, c);
    return r;
}
```

如果是二维棋盘版本（LC 723 粉碎糖果）：每轮标记横向/纵向连续 ≥3 个的格子 → 清除 → 按列下落 → 重复，直到没有可消除的格子。

9. 只出现一次的数字（LC 136 → 137）

```cpp
int single136(vector<int>& a) { int r = 0; for (int x : a) r ^= x; return r; }
// 137：一个数出现一次，其他数出现三次 → 统计每一位上 1 的个数，再模 3
int single137(vector<int>& a) {
    int r = 0;
    for (int b = 0; b < 32; b++) {
        int c = 0; for (int x : a) c += (x >> b) & 1;
        if (c % 3) r |= (1u << b);
    }
    return r;
}
```

面经里的变种是“有一个数出现两次，其他数都出现三次”：同样按位统计，某一位上的计数 % 3 == 2，说明目标数在这一位上是 1。

10. 编辑距离（LC 72）

```
int minDistance(string a, string b) {
    int n = a.size(), m = b.size();
    vector<vector<int>> dp(n + 1, vector<int>(m + 1));
    for (int i = 0; i <= n; i++) dp[i][0] = i;
    for (int j = 0; j <= m; j++) dp[0][j] = j;
    for (int i = 1; i <= n; i++)
        for (int j = 1; j <= m; j++)
            dp[i][j] = a[i-1] == b[j-1] ? dp[i-1][j-1]
                     : 1 + min({dp[i-1][j], dp[i][j-1], dp[i-1][j-1]});
    return dp[n][m];
}
```

11. Rand5 → Rand7 / Rand3（LC 470）

```
int rand5();                          // 等概率返回 0..4
int rand7() {
    while (true) {
        int x = rand5() * 5 + rand5(); // 等概率得到 0..24
        if (x < 21) return x % 7;
    }
}
int rand3() { int x; while ((x = rand5()) >= 3); return x; }
```

注意：面经中那位同学写的 `(rand5()*6 + rand5() + 1)/10` **不是等概率的**，面试时要用拒绝采样。

12. 非递归快排

```
int partition(vector<int>& a, int l, int r) {
    swap(a[l + rand() % (r - l + 1)], a[r]);   // 随机选主元，避免退化
    int p = a[r], i = l;
    for (int j = l; j < r; j++) if (a[j] < p) swap(a[i++], a[j]);
    swap(a[i], a[r]);
    return i;
}
void quickSort(vector<int>& a) {
    if (a.empty()) return;
    stack<pair<int,int>> st; st.push({0, (int)a.size() - 1});
    while (!st.empty()) {
        auto [l, r] = st.top(); st.pop();
        if (l >= r) continue;
        int m = partition(a, l, r);
        st.push({l, m - 1}); st.push({m + 1, r});
    }
}
```

13. 阻塞队列 / 生产者-消费者

```cpp
template<class T> class BlockingQueue {
    queue<T> q; size_t cap; mutex mu; condition_variable notFull, notEmpty;
public:
    explicit BlockingQueue(size_t c): cap(c) {}
    void push(T x) {
        unique_lock<mutex> lk(mu);
        notFull.wait(lk, [&]{ return q.size() < cap; });   // 带谓词的 wait 可以防止虚假唤醒
        q.push(move(x));
        notEmpty.notify_one();
    }
    T pop() {
        unique_lock<mutex> lk(mu);
        notEmpty.wait(lk, [&]{ return !q.empty(); });
        T x = move(q.front()); q.pop();
        notFull.notify_one();
        return x;
    }
};
```

Go 版本直接用 `make(chan T, cap)`。追问“怎么优化互斥锁”：缩小临界区、读写锁、分段锁、无锁队列（CAS 环形队列）、批量处理、先自旋再休眠。

14. 大数乘法（LC 43）

```cpp
string multiply(string a, string b) {
    int n = a.size(), m = b.size();
    vector<int> r(n + m, 0);
    for (int i = n - 1; i >= 0; i--)
        for (int j = m - 1; j >= 0; j--) {
            int s = (a[i] - '0') * (b[j] - '0') + r[i + j + 1];
            r[i + j + 1] = s % 10; r[i + j] += s / 10;
        }
    string s; for (int d : r) if (!(s.empty() && d == 0)) s += char('0' + d);
    return s.empty() ? "0" : s;
}
```

15. 交替打印奇偶数（Go）

```go
func main() {
    odd, even, done := make(chan struct{}), make(chan struct{}), make(chan struct{})
    go func() { for i := 1; i <= 9; i += 2 { <-odd; fmt.Println(i); even <- struct{}{} } }()
    go func() {
        for i := 2; i <= 10; i += 2 {
            <-even; fmt.Println(i)
            if i == 10 { close(done); return }
            odd <- struct{}{}
        }
    }()
    odd <- struct{}{}
    <-done
}
```

16. 设计 0~10 万整数的集合，CRUD 和遍历都要快（CSIG 二面）

```cpp
// sparse set：所有操作 O(1)，遍历只走实际存在的元素
struct IntSet {
    vector<int> dense, pos = vector<int>(100001, -1);
    bool has(int x) { return pos[x] != -1; }
    void add(int x) { if (!has(x)) { pos[x] = dense.size(); dense.push_back(x); } }
    void del(int x) {
        if (!has(x)) return;
        int last = dense.back();
        dense[pos[x]] = last; pos[last] = pos[x];   // 用末尾元素覆盖被删元素的位置
        dense.pop_back(); pos[x] = -1;
    }
    // 遍历：for (int x : dense)
};
```

面试官否定的方案：只用 map（他认为遍历时要扫描空位）、只用数组（删除慢）。这道题的关键是“**下标数组 + 紧凑数组 + 删除时与末尾元素交换**”。如果要求遍历有序，可以用 bitmap + 分层索引（例如 van Emde Boas 或两级 bitmap）。

17. 判断点集能否构成凸多边形（只说思路）

- 如果点**已经按顺序给出**：依次计算相邻两条边的叉积 `cross(p[i+1]-p[i], p[i+2]-p[i+1])`，所有叉积同号就是凸多边形。
- 如果点**是无序的**：用 Andrew 单调链算法求凸包，O(n log n)，判断凸包上的点数是否等于 n（共线的点按题意处理）。

四、复习建议

1. **链表专题**一定要练熟（21/23/24/25/206/92/141/142/143/148/160/19），腾讯云面试中链表出现得最多。
2. **LRU**：手写双向链表版本 + 并发分片 + 分布式扩展的口述，作为一整套准备。
3. 练习 **ACM 模式**的输入输出，以及在限时 20 分钟内写完并自己编测试用例。
4. 二分查找要能在不看资料的情况下写对左闭右开的 lower_bound。
5. 二面会出口算题、设计题，按面试官的要求作答（有面经吐槽“面试官要求口述，候选人坚持手写被扣分”）。